



AI Travel Planning Assistant Travito

**Aira Salman
23108103
BSAI 6A**

Developer Installation

Project: AI Travel Planning Assistant (Travito)

This guide provides instructions to compile, configure, execute, and navigate the AI Travel Planning Assistant — covering environment setup, backend/frontend installation, and a full walkthrough of the end-user interface.

1. System Requirements & Prerequisites

- Node.js — Version 18.x or above
- npm — Version 9.x or above
- SQLite — Local SQL engine (binary downloaded automatically by the project's dependency wrapper better-sqlite3)

2. Developer Installation Steps

Step 2.1 — Clone or Open Workspace

Ensure the root directory contains the following components:

```
├── package.json
├── vite.config.js
├── index.html
├── src/
├── backend/
│   ├── package.json
│   └── server.js
├── database/
│   └── schema.sql
```

Step 2.2 — Install Dependencies

Open terminal windows in the project folder and compile local modules.

1. Frontend (Root Folder):

```
npm install
```

2. Backend (API Server Folder):

```
# Make sure to set the workspace prefix or navigate to backend
npm --prefix backend install
```

Step 2.3 — Environment Configuration

Create a .env configuration file under backend/.env. It should resemble the following:

```
PORT=5001
```

```
JWT_SECRET=super_secret_travel_key_123  
GROQ_API_KEY=YOUR_GROQ_API_KEY
```

NOTE

If GROQ_API_KEY is not supplied or the variable is omitted, the API server will launch in Mock Fallback Mode, allowing evaluations of all routes, itinerary creation, and chatbot interfaces immediately without requiring active keys.

Step 2.4 — Execution

Start both servers together using the concurrently command in the root workspace folder:

```
npm run start:all
```

This command runs:

- Vite React Frontend at <http://localhost:3000>
- Node.js Express Backend at <http://localhost:5001>

3. User Navigation Guide

Step 3.1 — Security Gate (Authentication)

- On launching the app, users are greeted with a glassmorphic authorization card.
- New Users: Click “Create an Account”, input name, email, and password, and click “Create Account”.
- Returning Users: Login with registered email and password.
- Credentials are automatically saved, bypassing the gate on subsequent sessions using JWT.

Step 3.2 — The Travel Dashboard

- View travel stats: number of active itineraries, total planned budget limits, and upcoming destinations.
- Scroll through active trip cards.
- Click a card to view day-by-day schedules, hotels, packing guidelines, and active budgets.
- Delete a trip card by clicking the trash icon.

Step 3.3 — Planning a Trip via AI

- Click “Plan New Trip” in the navigation sidebar.
- Input destination name, start/end dates, budget cap limit, travel styles (Culture, Adventure, Relaxation, Foodie), and companions.
- Click “Generate AI Draft”.
- The AI creates a timeline draft. Review the schedule, hotel ratings, and packing suggestions.
- Click “Save Trip” to commit the plan to your active SQLite travel database.

Step 3.4 — Dynamic Packing Checklist

- Click on the “Packing List” tab inside an active trip detail.
- Items are categorized (Documents, Clothing, Electronics, etc.).
- Click items to mark them as packed. Checked items are saved.

Step 3.5 — Expense Composition Tracker

- Click the “Expenses & Budget” tab to check financial indicators.
- View the circular exhaustion gauge showing what percentage of the budget is spent.
- Stroll through custom categories progress bars.
- Log new expenses: enter Description, Amount, Category, and Date, then click “Add Expense”.
- Delete errors by clicking the trash bin icon next to any history entry.

Step 3.6 — AI Chat Assistant Sidebar

- Click “AI Travel Guide” tab.
- Type regional questions (e.g. “is the tap water drinkable?”, “should I tip at restaurants?”) and click send.
- The chatbot replies using local context.

Quick Reference

Component	URL / Value
Frontend (Vite + React)	http://localhost:3000
Backend (Node.js + Express)	http://localhost:5001
Default Port (backend)	5001
Database	SQLite (via better-sqlite3)
Fallback Mode	Activated automatically if GROQ_API_KEY is missing